

תשובות לשאלות:

1) כשאנחנו קוראים ל-`fork()` בתוכנית, מתבצעת פסיקת תוכנה שגורמת למעבד לעצור את הריצה הנוכחית ולשמור את מצב האוגרים. בשלב זה, המעבד עובר מ-`User Mode` ל-`Kernel Mode`. הקרנל ניגש לטבלת וקטור הפסיקות (`Interrupt Vector Table`) כדי להריץ את קוד הפסיקה של `fork`. הקרנל יוצר בלוק בקרה (`PCB`) חדש לבן, משכפל את טבלאות הדפים של האב (תוך הגדרתם כ-`Copy-on-Write`) ומקצה לו `PID` חדש. לבסוף, המעבד חוזר ל-`User Mode` והתוכנית ממשיכה לרוץ, כשהאב מקבל את ה-`PID` של הבן והבן מקבל 0.

2) בתהליך הכתיבה ללוג שלנו יש 3 שכבות של חוצצים: זיכרון במרחב המשתמש (כמו ב-`printf`, למרות שפה השתמשנו ישירות ב-`write`), ה-`Page Cache` של הקרנל, והחוצץ הפיזי של הדיסק עצמו. אם התוכנית שלנו קורסת פתאום, עסקאות שכבר שלחנו עם `write()` ונמצאות ב-`Page Cache` אבל עוד לא נכתבו פיזית לדיסק (`Flush`) ילכו לאיבוד. כדי למנוע את זה ולהבטיח שהמידע נשמר על הדיסק אפשר להשתמש ב-`fsync()`, אבל זה פוגע מאוד בביצועים כי זה חוסם את התהליך עד שפעולת ה-`I/O` האיטית מסתיימת.

3) מיד אחרי ה-`fork()`, תהליכי האב והבן שלנו מצביעים לאותם דפי זיכרון פיזיים שמוגדרים לקריאה-בלבד, כדי לחסוך משאבים. בזכות מנגנון ה-`Copy-on-Write`, ברגע שתהליך מנסה לכתוב (למשל, לשנות יתרה), הקרנל תופס את שגיאת הדף (`Page Fault`), מעתיק את הדף למיקום חדש, ורק אז מאפשר את הכתיבה. לכן, כשסניף בן מעדכן יתרת חשבון, הוא עושה את זה על עותק פרטי משלו, ועותק האב בכלל לא משתנה. זו בדיוק הסיבה שבגללה היינו חייבים להשתמש בצינורות (`Pipes`) כדי להעביר את הסיכומים חזרה לאב.

4) תנאי תחרות מתרחש כי פעולה כמו `amount += balance` היא לא אטומית ומתורגמת ל-3 פקודות מכונה: קריאה לאוגר (`LOAD`), הוספה (`ADD`), וכתיבה חזרה (`STORE`). לדוגמה, אם חוט א' קורא יתרה של 100 ש"ח (`LOAD`) ואז יש `Context Switch`. חוט ב' רץ, קורא את אותם 100 ש"ח, מוסיף 50 וכותב חזרה 150 (`STORE`). עכשיו חוט א' חוזר לרוץ, מוסיף 20 ל-100 שעדיין שמור לו באוגר, וכותב 120. ככה ההפקדה של חוט ב' נדרסה ונעלמה! ה-`Mutex` שהוספנו מונע את זה כי הוא מגדיר את העדכון כקטע קריטי: כשחוט אחד נועל את החשבון, חוט אחר חייב לחכות עד שהראשון יסיים את כל שלוש הפקודות, וכך אין דריסות.

5) מנעול ה-`pthread_mutex_t` שבחרנו לא משתמש בהמתנה פעילה (`Busy Wait`) - אם המנעול תפוס, החוט פשוט "נרדם" ולא מבזבז מעבד. לעומתו, `Spin Lock` כן עושה המתנה פעילה ומסתובב בלולאה עד שהמנעול משתחרר. לכן, `Spin Lock` מתאים מאוד לאזורים קריטיים קצרים במערכת מרובת ליבות (כי הוא חוסך את זמן ההרדמה וההערה), אבל הוא ממש לא מתאים לאזורים ארוכים כמו `I/O`, כי הוא סתם שורף משאבי `CPU` בזמן ההמתנה. למטלה הזו העדפתי להשתמש ב-`Mutex`, גם כי זה נדרש וגם כי הפקידים שלנו מבצעים פעולות נוספות שהיו הופכות את ה-`Spin Lock` ללא יעיל.

6) לא השתמשנו ב-`printf()` בתוך `signal handler` כי היא לא מוגדרת כ-`async-signal-safe` המשמעות של המונח היא שהפונקציה בטוחה לשימוש גם אם היא נקטעת על ידי איתות ונקראת שוב מתוכו. `printf` משתמשת בחוצצים ומנעולים פנימיים, ואם האיתות קוטע אותה כשהמנעול תפוס, ניסיון לקרוא לה שוב יגרום לקיפאון. פונקציות כמו `write()` ו-`write()` הן `async-signal-safe`.

`waitpid()` הן כן בטוחות (כי הן קריאות מערכת ישירות), בעוד ש-`printf()` ו-`malloc()` אינן בטוחות מאותה סיבה של ניהול מצב גלובלי ומנעולים.

(7) כשאנחנו עושים `write()` לצינור, הנתונים נכתבים לתוך חוצץ (Buffer) פנימי בזיכרון של מערכת ההפעלה, ולא לדיסק. אם האב מנסה לעשות `read()` לפני שאיזשהו ילד כתב, הקרנל פשוט יחסום (Block) את האב וירדים אותו עד שיהיו נתונים זמינים. חשוב מאוד שהאב יסגור את צד הכתיבה שלו, כי קריאת `read` מסתיימת (ומחזירה 0) רק כשאינ יותר כותבים פוטנציאליים לצינור. אם האב ישאיר את הצד שלו פתוח, ה-`read` שלו לעולם לא יסתיים והתוכנית תיתקע לנצח.

(8) כשאנחנו קוראים ל-`malloc()` להקצאת זיכרון, הוא מגיע מסגמנט הערימה (Heap) של התהליך. עם זאת, הקרנל משתמש ב-Demand Paging ולא מקצה זיכרון פיזי מיד. בפעם הראשונה שניגש לדף הזה, תיווצר פסיקת שגיאת דף (Page Fault). הקרנל יתפוס אותה, יקצה דף פיזי פנוי, ימפה אותו לדף הווירטואלי, ויחזיר את התוכנית להמשך ריצה. במצב של לחץ זיכרון שדורש החלפת דפים, מערכת ההפעלה כנראה תשתמש באלגוריתם ה-Clock (קירוב של LRU). אלגוריתם LRU אמיתי דורש עדכון חותמת זמן על כל גישה לזיכרון וזה כבד מדי, בעוד שה-Clock רק בודק Reference Bit שמעודכן ביעילות על ידי החומרה.

Race Condition Demonstration

הערה: כדי באמת להדגים את תנאי התחרות כמו שצריך, הרצתי את הבדיקות עם עומס של 100,000 פעולות במקום 20. הסיבה היא ש-20 פעולות מסתיימות כל כך מהר על מעבד מודרני, שהחוסים בכלל לא מספיקים לחפוף ולהתנגש אחד בשני. הגדלת העומס הכריחה את החוסים לרוץ במקביל באותו זמן ולדרוס אחד את השני.

הרצה עם מנעולים:

```
PS C:\Users\Noa Bouchnick\Desktop\מנהלי-סיווג\מסמך-1\מסמך-1\PROG_3> docker compose run -- N_TX=10
0000 bank sh -c "make clean && make && ./bank"
Loading accounts:
ID      Owner      Balance
-----
1001    Alice      $5000
1002    Bob         $3200
1003    Carol       $8750
1004    David       $1500
1005    Eve         $9999
>>> Branch 1 Race Condition Check: PASSED
>>> Branch 0 Race Condition Check: PASSED

=== Final Bank Report ===
Branch 0: 500000 transactions | +62720037 deposited | -6226419 withdrawn
Branch 1: 500000 transactions | +62521902 deposited | -62303181 withdrawn
>>> Branch 2 Race Condition Check: PASSED
Branch 2: 500000 transactions | +62700097 deposited | -62367813 withdrawn
Total: 1500000 transactions | +187942036 deposited | -186897413 withdrawn

Account balances after all transactions:
1001 Alice: $5000
1002 Bob: $3200
1003 Carol: $8750
1004 David: $1500
1005 Eve: $9999

Balance conservation check: PASSED (initial sum 28449 == final sum 28449)
```

ריצה תקינה (עם מנעולים): כשהמנעולים עובדים, כל פקיד (חוט) נועל את החשבון לפני שהוא קורא ומשנה אותו. בפלט רואים שכל הסניפים הדפיסו PASSED בבדיקת חוק שימור הכסף - אף חוט לא נכנס באמצע הפעולה של חוט אחר, והסכום תמיד נשאר מאוזן ומדויק.

הרצה ללא מנעולים:

```

PS C:\Users\Woa Bouchnick\Desktop> gcc -std=c11 -pthread -DNO_LOCK -o bank bank.c
0000 bank sh -c "make clean && make race && ./bank"
gcc -Wall -Wextra -pthread -DNO_LOCK -o bank bank.c
Loading accounts:
ID      Owner      Balance
-----
1001    Alice      $5000
1002    Bob        $3200
1003    Carol      $8750
1004    David      $1500
1005    Eve        $9999
>>> Branch 1 Race Condition Check: FAILED (Expected 482277, Got 483959)
>>> Branch 0 Race Condition Check: FAILED (Expected 228079, Got 227473)

=== Final Bank Report ===
Branch 0: 500000 transactions | +62683193 deposited | -62483563 withdrawn
Branch 1: 500000 transactions | +62595413 deposited | -62141585 withdrawn
>>> Branch 2 Race Condition Check: FAILED (Expected 275255, Got 275089)
Branch 2: 500000 transactions | +62701565 deposited | -62454759 withdrawn
Total: 1500000 transactions | +187980171 deposited | -187079907 withdrawn

Account balances after all transactions:
1001 Alice: $5000
1002 Bob: $3200
1003 Carol: $8750
1004 David: $1500
1005 Eve: $9999

Balance conservation check: PASSED (initial sum 28449 == final sum 28449)

```

ריצה בלי מנעולים(NO_LOCK): פה ביטלתי את המנעולים בזמן הקימפול. בגלל העומס, החוטים קראו וכתבו לאותם חשבונות בדיוק באותו זמן. זה גרם ל"חיתוך" של הפקודות ברמת המעבד ולדריסת מידע (למשל חוט אחד דרס הפקדה של חוט אחר). בגלל זה הסניפים הדפיסו FAILED - הסכום שהיה אמור להיות בסוף לא תאם לסכום בפועל, וכסף פשוט "נעלם".

בשתי הריצות, תהליך האב בסוף התוכנית תמיד הדפיס PASSED עם הסכום ההתחלתי. זה קורה כי אחרי שעשינו fork, האב בכלל לא רואה את השינויים של הסניפים בזכות מנגנון ה-Copy-on-Write. הסניפים עשו את הבלאגן על העותק הפרטי שלהם, והמערך המקורי של האב נשאר שלם בדיוק כמו שהוא היה בהתחלה.

בונוס א' - בקרת עומסים עם סמפור: כדי למנוע מהשרת לקרוס או להיתקע מעומס, הוספתי סמפור מונה (Counting Semaphore) שהגדרתי מראש ל-MAX_ACTIVE (שהוא 2). כל סניף עושה sem_wait לפני שהוא מתחיל ליצור את החוטים שלו, ו-sem_post רק אחרי שכל החוטים סיימו לעבוד. במקום שכל הסניפים ירצו במקביל, ילחמו על המעבד ויעשו המון קריאות I/O מול הדיסק, הסמפור מוודא שרק 2 סניפים עובדים בכל רגע נתון. שאר הסניפים שניגשים מורדמים (Blocked) על ידי מערכת ההפעלה ולא מבזבזים לנו משאבי מעבד בכלל עד שמגיע תורם.